Movie Recommendation System

Here [MovieLens 20M Dataset]() by `GroupLens` is used to build `content based` and `collaborative filtering` recommendation systems.

```
from math import sqrt

import pandas as pd
```

```
# Pandas config
def pandas_config():
    # display 10 rows and all the columns
    pd.set_option('display.max_rows', 20)
    pd.set_option('display.max_columns', None)


pandas_config()
```

```
# Global path variables
RATINGS_PATH = '/kaggle/input/movielens-20m-dataset/rating.csv'
LINK_PATH = '/kaggle/input/movielens-20m-dataset/link.csv'
GENOME_PATH = '/kaggle/input/movielens-20m-dataset/genome_tags.csv'
GENOME_SCORES_PATH = '/kaggle/input/movielens-20m-dataset/genome_scores.csv'
TAGS_PATH = '/kaggle/input/movielens-20m-dataset/tag.csv'
MOVIE_PATH = '/kaggle/input/movielens-20m-dataset/movie.csv'
```

```
movies_df = pd.read_csv(MOVIE_PATH)
ratings_df = pd.read_csv(RATINGS_PATH)
```

```
movies_df.sample(5)
```

	movieId	title	genres
17094	86541	To the Sea (Alamar) (2009)	Drama
10153	33683	High Tension (Haute tension) (Switchblade Roma...	Horror|Thriller
4927	5023	Waterdance, The (1992)	Drama
1877	1961	Rain Man (1988)	Drama
12775	60229	Unholy Three, The (1930)	Crime|Drama

```
ratings_df.sample(5)
```

	userId	movieId	rating	timestamp
3185161	21755	44	4.0	1996-06-21 13:01:03
14498697	100203	282	5.0	2005-04-08 14:47:06
1125606	7678	329	2.0	1996-10-19 03:52:49
4109530	27949	1375	4.0	1999-01-19 00:17:43
1971903	13371	63082	4.5	2009-06-14 05:42:55

🎻 Data Preparation

Cleaning the `movie_df`

```
# Remove the years from the title of `movies_df`
def rm_dates_from_title(df: pd.DataFrame):
    # Using regular expressions to find a year stored between parentheses

    # We specify the parantheses so we don't conflict with movies that have years in their titles
    df['year'] = df.title.str.extract('(\(\d\d\d\d\))', expand=False)

    # Removing the parentheses
    df['year'] = df.year.str.extract('(\d\d\d\d)', expand=False)

    # Removing the years from the 'title' column
    df['title'] = df.title.str.replace('(\(\d\d\d\d\))', '', regex=True)

    # Applying the strip function to get rid of any ending whitespace characters that may have appeared
    df['title'] = df.title.apply(lambda x: x.strip())


rm_dates_from_title(movies_df)
movies_df.sample(5)
```

	movieId	title	genres	year
6851	6963	Devil's Playground	Documentary	2002
26524	127204	The Overnight	Comedy	2015
21002	102481	Internship, The	Comedy	2013
22041	106226	Clandestine Childhood	Drama	2011
24636	116507	Mission London	Comedy	2010

```
# Every genre is separated by a | so we simply have to call the split function on |
movies_df.genres = movies_df.genres.str.split('|')
movies_df.sample(5)
```

	movieId	title	genres	year
1717	1789	Sadness of Sex, The	[Drama]	1995
722	734	Getting Away With Murder	[Comedy]	1996
7198	7310	Raw Deal	[Action]	1986
990	1009	Escape to Witch Mountain	[Adventure, Children, Fantasy]	1975
25475	120534	A Master Builder	[Drama]	2013

Since keeping genres in a list format isn't optimal for the content-based recommendation system technique, we will use the `One Hot Encoding` technique to convert the list of genres to a vector where each column corresponds to one possible value of the feature.

This encoding is needed for feeding categorical data. In this case, we store every different genre in columns that contain either 1 or 0. `1 shows that a movie has that genre and 0 shows that it doesn't`. Let's also store this dataframe in another variable since genres won't be important for our first recommendation system

```
# one-hot-encode movies_df's genres column
def one_hot_encode_genres(df: pd.DataFrame):
    # Copying the movie dataframe into a new one since we won't need to use
    # the genre information in our content-based recommendation system.
    movies_with_genres_df = df.copy()

    # For every row in the dataframe, iterate through the list of genres and place
    # a 1 into the corresponding column
    for index, row in df.iterrows():
        for genre in row.genres:
            movies_with_genres_df.at[index, genre] = 1

    # Filling in the NaN values with 0 to show that a movie doesn't have that column's genre
    movies_with_genres_df.fillna(0, inplace=True)
```

```
    return movies_with_genres_df


movies_with_genres_df = one_hot_encode_genres(movies_df)
movies_with_genres_df.sample(5)
```

	movieId	title	genres	year	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thrill
21803	105364	Something in the Air (Apres Mai)	[Action, Drama]	2012	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0
19434	96496	6 Bullets	[Action, Crime, Thriller]	2012	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1
12658	59519	Reprise	[Drama]	2006	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0
20083	98963	Neighbouring Sounds (O som ao redor)	[Drama, Thriller]	2012	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1
23107	110173	Wolf	[Crime, Drama, Thriller]	2013	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	1

Cleaning the `rating_df`

```
ratings_df.sample(5)
```

	userId	movieId	rating	timestamp
2102763	14231	364	3.0	2005-10-29 12:20:15
10962248	75810	3727	4.0	2009-09-19 07:33:26
3175496	21677	508	3.5	2008-05-07 16:02:25
8503341	58720	5323	2.5	2005-12-25 23:03:11
1284760	8751	2692	4.0	2004-02-15 15:44:13

Every row in the ratings dataframe has a user id associated with at least one movie, a rating and a timestamp showing when they reviewed it. We won't be needing the `timestamp column`, so let's drop it to save on memory.

```
# Drop removes a specified row or column from a dataframe
ratings_df.drop('timestamp', axis='columns', inplace=True)
ratings_df.sample(5)
```

	userId	movieId	rating
10172883	70379	1291	4.5
7160381	49355	31694	4.0
13354652	92290	3500	4.0
3728946	25383	6250	2.0
6549870	45027	1615	3.5

```
# Creating an input user to recommend movies to
def get_dummy_user():
    '''This will the user to whome we will recommend movies. This user has his own list
      of favourite movies which we will use to recommend him movies related to those
      movies which he has highly rated.

      Notice: To add more movies, simply increase the amount of elements in the userInput.
      Feel free to add more in! Just be sure to write it in with capital letters and if a
      movie starts with a "The", like "The Matrix" then write it in like this: 'Matrix, The'. '''
```

```
    user_input = [
        {
            'title': 'Breakfast Club, The',
            'rating': 5
        }, {
            'title': 'Toy Story',
            'rating': 3.5
        }, {
            'title': 'Jumanji',
            'rating': 2
        }, {
            'title': "Pulp Fiction",
            'rating': 5
        }, {
            'title': 'Akira',
            'rating': 4.5
        }
    ]

    input_movies = pd.DataFrame(user_input)
    return input_movies


input_movies = get_dummy_user()
input_movies.sample(5)
```

	title	rating
2	Jumanji	2.0
0	Breakfast Club, The	5.0
4	Akira	4.5
3	Pulp Fiction	5.0
1	Toy Story	3.5

```
# Add movieId to input user
def add_movies_ids(movies_df, input_movies):
    # Filtering out the movies by title
    input_id = movies_df[movies_df.title.isin(input_movies.title.tolist())]

    # Then merging it so we can get the movie_id. It's implicitly merging it by title.
    input_movies = pd.merge(input_movies, input_id)

    # Dropping movies information that we won't use from the input dataframe
    input_movies.drop(['genres', 'year'], axis='columns', inplace=True)

    return input_movies


input_movies = add_movies_ids(movies_df, input_movies)
input_movies.sample(5)
```

	title	rating	movieId
2	Jumanji	2.0	2
0	Breakfast Club, The	5.0	1968
1	Toy Story	3.5	1
4	Akira	4.5	1274
3	Pulp Fiction	5.0	296

`input_movies`, `rating_df` and `movie_df` are not altered in either recommendation systems

🎯 Building content based recommendation system

`Content-Based` or `Item-Item recommendation systems`, this technique attempts to figure out what a user's favourite aspects of an item is and then recommends items that present those aspects.

Here we're going to try to figure out the input's `favourite genres` from the `movies` and `ratings` given.

We're going to start by learning the `input's preferences`, so let's get the subset of movies that the input has watched from the Dataframe containing genres defined with binary values.

```
# Filtering out the movies from movies_with_genres_df
user_movies = movies_with_genres_df[movies_with_genres_df['movieId'].isin(
    input_movies['movieId'].tolist()
)]

user_movies.sample(5)
```

	movieId	title	genres	year	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thriller
1246	1274	Akira	[Action, Adventure, Animation, Sci-Fi]	1988	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
1884	1968	Breakfast Club, The	[Comedy, Drama]	1985	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0
0	1	Toy Story	[Adventure, Animation, Children, Comedy, Fantasy]	1995	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
1	2	Jumanji	[Adventure, Children, Fantasy]	1995	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
293	296	Pulp Fiction	[Comedy, Crime, Drama, Thriller]	1994	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0

```
# We'll only need the actual genre table, so let's clean this up a bit by resetting the
# index and dropping the movieId, title, genres and year columns.

# Resetting the index to avoid future issues
user_movies = user_movies.reset_index(drop=True)

# Dropping unnecessary issues due to memory and to avoid issues
user_genre_df = user_movies.drop(['movieId', 'title', 'genres', 'year'], axis='columns')

print(user_genre_df.shape)
user_genre_df
```

```
(5, 20)
```

	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thriller	Horror	Mystery	Sci-Fi	IMAX	Document
0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
1	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	
3	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	

Now we're ready to start learning the input's preferences!

To do this, we're going to `turn each genre into weights`. We can do this by using the input's reviews and multiplying them into the input's genre table and then summing up the resulting table by column. This operation is actually a `dot product between a matrix and a vector`.

```
input_movies['rating']
```

```
0    5.0
1    3.5
2    2.0
3    5.0
4    4.5
Name: rating, dtype: float64
```

```
# Dot product to get weights
user_profile = user_genre_df.T.dot(input_movies['rating'])

print(user_profile.shape)
user_profile.head(len(user_profile))
```

```
(20,)
Adventure             13.5
Animation             10.0
Children               8.5
Comedy                11.5
Fantasy                8.5
Romance                0.0
Drama                  6.5
Action                 5.0
Crime                  2.0
Thriller               2.0
Horror                 0.0
Mystery                0.0
Sci-Fi                 5.0
IMAX                   0.0
Documentary            0.0
War                    0.0
Musical                0.0
Western                0.0
Film-Noir              0.0
(no genres listed)     0.0
dtype: float64
```

In this dot product we understand that what user likes, so we see what is the `combined adventure, romance, etc...` does users rated movies have & from there we understand that the user likes adventure movies alot, the user's second preference is romantic movies and so on... This is what the weights tells us.

Now, we have the `weights for every genre of the user's preferences`. This is known as the `User Profile`. Using this, we can recommend movies that satisfy the user's preferences.

```
# Now let's get the genres of every movie in our original dataframe
genre_df = movies_with_genres_df.set_index(movies_with_genres_df['movieId'])

# Droping the unnecessary information
genre_df.drop(['movieId', 'title', 'genres', 'year'], axis='columns', inplace=True)

print(genre_df.shape)
genre_df.sample(5)
```

```
(27278, 20)
```

movieId	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thriller	Horror	Mystery	Sci-Fi	IMAX	Do
121152	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
104175	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	
95311	0.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
6323	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0	

With the `input's profile` and the `complete list of movies and their genres` in hand, we're going to take the `weighted average of every movie based on the input profile` and recommend the top twenty movies that most satisfy it.

Below are the few info about `genre_df` and `user_profile` to understand the recommendation code logic

```
print(genre_df.shape)
print(user_profile.shape)
```

```
(27278, 20)
(20,)
```

```
user_profile
```

```
Adventure             13.5
Animation             10.0
Children               8.5
Comedy                11.5
Fantasy                8.5
Romance                0.0
Drama                  6.5
Action                 5.0
Crime                  2.0
Thriller               2.0
Horror                 0.0
Mystery                0.0
Sci-Fi                 5.0
IMAX                   0.0
Documentary            0.0
War                    0.0
Musical                0.0
Western                0.0
Film-Noir              0.0
(no genres listed)     0.0
dtype: float64
```

```
genre_df.head(2)
```

movieId	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thriller	Horror	Mystery	Sci-Fi	IMAX	Do
1	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	

```
genre_df.head(2) * user_profile
```

movieId	Adventure	Animation	Children	Comedy	Fantasy	Romance	Drama	Action	Crime	Thriller	Horror	Mystery	Sci-Fi	IMAX	Do
1	13.5	10.0	8.5	11.5	8.5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	

```
(genre_df.head(2) * user_profile).sum(axis='columns')
```

```
movieId
1    52.0
2    30.5
dtype: float64
```

The above gives how much the `input user` will like movie with `movieId` 1 & 2. We can say that user might like movie with movieId 1 more than 2.

Below is `get_recommendation_df` applys the above recommendation logic over the entire `genre_df` dataset.

```
# Multiplying each row in genre_df with the user_profile and summing that row values
# to get wieght to recommend the movies
def get_recommendation_df(genre_df, user_profile):
    # Also normalizing the values by dividing by user_profile.sum()
    df = ((genre_df * user_profile).sum(axis='columns')) / user_profile.sum()
    return df


recommendation_df = get_recommendation_df(genre_df, user_profile)

print(recommendation_df.shape)
recommendation_df.head()
```

```
(27278,)
movieId
1    0.717241
2    0.420690
3    0.158621
4    0.248276
5    0.158621
dtype: float64
```

```
# Sort our recommendations in descending order
recommendation_df = recommendation_df.sort_values(ascending=False)
recommendation_df.head()
```

```
movieId
26093    0.806897
51632    0.786207
51939    0.786207
673      0.786207
26340    0.786207
dtype: float64
```

Top 20 recommendations

```
# Final recommedation table
# Getting only top 20 movies to recommend to user
movies_df.loc[movies_df['movieId'].isin(recommendation_df.head(20).keys())]
```

	movieId	title	genres	year
664	673	Space Jam	[Adventure, Animation, Children, Comedy, Fanta...	1996
2901	2987	Who Framed Roger Rabbit?	[Adventure, Animation, Children, Comedy, Crime...	1988
4211	4306	Shrek	[Adventure, Animation, Children, Comedy, Fanta...	2001
8603	26093	Wonderful World of the Brothers Grimm, The	[Adventure, Animation, Children, Comedy, Drama...	1962
8780	26340	Twelve Tasks of Asterix, The (Les douze travau...	[Action, Adventure, Animation, Children, Comed...	1976
9291	27344	Revolutionary Girl Utena: Adolescence of Utena...	[Action, Adventure, Animation, Comedy, Drama, ...	1999
9819	32031	Robots	[Adventure, Animation, Children, Comedy, Fanta...	2005
10114	33463	DuckTales: The Movie - Treasure of the Lost Lamp	[Adventure, Animation, Children, Comedy, Fantasy]	1990
10367	36397	Valiant	[Adventure, Animation, Children, Comedy, Fanta...	2005
10565	40339	Chicken Little	[Action, Adventure, Animation, Children, Comed...	2005
11704	51632	Atlantis: Milo's Return	[Action, Adventure, Animation, Children, Comed...	2003
11739	51939	TMNT (Teenage Mutant Ninja Turtles)	[Action, Adventure, Animation, Children, Comed...	2007
11773	52287	Meet the Robinsons	[Action, Adventure, Animation, Children, Comed...	2007
11794	52462	Aqua Teen Hunger Force Colon Movie Film for Th...	[Action, Adventure, Animation, Comedy, Fantasy...	2007
13091	62956	Futurama: Bender's Game	[Action, Adventure, Animation, Comedy, Fantasy...	2008
16851	85261	Mars Needs Moms	[Action, Adventure, Animation, Children, Comed...	2011
22048	106240	Free Birds	[Action, Adventure, Animation, Children, Comed...	2013
22763	108932	The Lego Movie	[Action, Adventure, Animation, Children, Comed...	2014
24460	115879	Toy Story Toons: Small Fry	[Adventure, Animation, Children, Comedy, Fantasy]	2011
24849	117454	The Magic Crystal	[Adventure, Animation, Children, Comedy, Fantasy]	2011

🎷 Building collaborative filtering recommendation system

`Collaborative Filtering`, which is also known as `User-User Filtering`.

As hinted by its alternate name, this technique uses other users to recommend items to the input user. It attempts to find users that have similar preferences and opinions as the input and then recommends items that they have liked to the input. There are several methods of finding `similar users` (Even some making use of Machine Learning), and the one used here is based on the `Pearson Correlation Function`.

The process for creating a `User Based recommendation` system is as follows:

- Select a user with the movies the user has watched
- Based on his rating to movies, find the top X neighbours
- Get the watched movie record of the user for each neighbour.
- Calculate a similarity score using some formula
- Recommend the items with the highest score

Getting the users who has `seen the same movies` as our input user. With the movie ID's in our input, we can now get the subset of users that have watched and reviewed the movies in our input.

```
user_subset = ratings_df[ratings_df['movieId'].isin(input_movies['movieId'].tolist())]

print(user_subset.shape)
user_subset.head()
```

```
(168730, 3)
```

	userId	movieId	rating
0	1	2	3.5
11	1	296	4.0
236	3	1	4.0
451	5	2	3.0
517	6	1	5.0

```
# We now group up the rows by user ID.
user_subset_group = user_subset.groupby(['userId'])

# let's look at one of the users, e.g. the one with userID=1130
user_subset_group.get_group(1130)
```

	userId	movieId	rating
166633	1130	1968	4.0

Let's also sort these groups so the users that share the `most movies in common` with the input have higher priority. This provides a richer recommendation since `we won't go through every single user`.

```
# Sorting users with movie most in common with the input will have priority
user_subset_group = sorted(user_subset_group, key=lambda x: len(x[1]), reverse=True)

# Top 3 users in user_subset_group
user_subset_group[0:3]
```

```
[(91,
        userId  movieId  rating
  9621      91        1     4.0
  9622      91        2     3.5
  9669      91      296     3.5
  9826      91     1274     2.5
  9903      91     1968     4.0),
 (294,
         userId  movieId  rating
  37452     294        1     4.5
  37453     294        2     4.5
  37504     294      296     4.5
  37648     294     1274     4.5
  37731     294     1968     5.0),
 (586,
          userId  movieId  rating
  81164     586        1     2.5
  81165     586        2     3.0
  81226     586      296     5.0
  81390     586     1274     4.0
  81499     586     1968     3.0)]
```

`Similarity of users to input user`

Next, we are going to compare all users (not really all !!!) to our specified user and find the one that is `most similar`. We're going to find out how similar each user is to the input through the `Pearson Correlation Coefficient`. It is used to measure the strength of a linear association between two variables.

Pearson correlation is `invariant to scaling`, i.e. multiplying all elements by a nonzero constant or adding any constant to all elements. For example, if you have two vectors X and Y,then, `pearson(X, Y) == pearson(X, 2 * Y + 3)`. This is a pretty `important property` in recommendation systems because for example two users might rate two series of items totally different in terms of absolute rates, but they would be similar users (i.e. with similar ideas) with similar rates in various scales.

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

The values given by the pearson correlation formula vary from r = -1 to r = 1, where 1 forms a direct correlation between the two entities (it means a perfect positive correlation) and -1 forms a perfect negative correlation. In our case, a 1 means that the two users have similar tastes while a -1 means the opposite.

We will select a subset of users to iterate through. This limit is imposed because we don't want to waste too much time going through every single user.

```
user_subset_group = user_subset_group[0:100]
```

Now, we calculate the Pearson Correlation between input user and subset group, and store it in a dictionary, where the key is the user Id and the value is the coefficient

```
def calculate_persona_corr(user_subset_group, input_movies):
    # Store the Pearson Correlation in a dictionary, where the key is the user Id and the
    # value is the coefficient
    pearson_corr_dict = {}

    # For every user group in our subset
    for name, group in user_subset_group:
        # Let's start by sorting the input and current user group so the values aren't mixed up later on
        group = group.sort_values(by='movieId')
        input_movies = input_movies.sort_values(by='movieId')

        # Get the N for the formula
        n_ratings = len(group)

        # Get the review scores for the movies that they both have in common
        temp_df = input_movies[input_movies['movieId'].isin(group['movieId'].tolist())]

        # And then store them in a temporary buffer variable in a list format to facilitate future calculations
        temp_rating_list = temp_df['rating'].tolist()

        # Let's also put the current user group reviews in a list format
        temp_group_list = group['rating'].tolist()

        # Now let's calculate the pearson correlation between two users, so called, x and y
        Sxx = sum([i**2 for i in temp_rating_list]) - pow(sum(temp_rating_list), 2) / float(n_ratings)
        Syy = sum([i**2 for i in temp_group_list]) - pow(sum(temp_group_list), 2) / float(n_ratings)
        Sxy = sum(i * j for i, j in zip(temp_rating_list, temp_group_list)) - sum(temp_rating_list) * sum(temp_group_list) / fl

        # If the denominator is different than zero, then divide, else, 0 correlation.
        if Sxx != 0 and Syy != 0:
            pearson_corr_dict[name] = Sxy / sqrt(Sxx * Syy)
        else:
            pearson_corr_dict[name] = 0

    return pearson_corr_dict


pearson_corr_dict = calculate_persona_corr(user_subset_group, input_movies)
pearson_corr_dict.items()
```

```
dict_items([(91, -0.08006407690254357), (294, 0.4385290096535115), (586, 0.5393193716300061), (648, 0.6880209161537812), (775,
0.8362420100070908), (812, 0.6016568375961869), (869, 0.1860521018838127), (903, -0.17902871850985827), (1200,
0.5370861555295743), (1244, 0.10963225241337883), (1715, 0.8951435925492911), (1748, 0.8320502943378437), (1763,
-0.268543077764787), (1810, 0.8594395636904102), (1813, 0.8347371386380908), (1849, 0.626600514784503), (1864,
0.8320502943378437), (1942, 0.774023530673004), (1984, -0.31803907173309875), (2047, 0.8976095575314932), (2099,
-0.4385290096535115), (2367, 0.49334513586020373), (2397, 0), (2515, 0.8951435925492914), (2661, 0.4385290096535153), (2757,
0.7844645405527362), (2959, 0.11720180773462363), (2988, 0.7197795937681559), (3179, 0.29417420270727607), (3218,
0.8503864129218268), (3268, 0.8204126541423654), (3269, 0.8648817040445187), (3318, 0.8790135580096794), (3397,
0.711233325153824), (3487, 0.36544084137792915), (3576, 0.5967623950328603), (3629, 0.8807048459279793), (3858,
```

```
0.4772135722318333), (3986, 0.6864064729836442), (4034, 0.657793514480273), (4332, 0.35805743701971815), (4358,
0.1386750490563073), (4529, 0.5765566601970551), (4598, 0.358057437019716), (4696, 0.6577935144802716), (5504,
-0.43852900965351443), (5518, 0.7205766921228921), (5576, 0.8807048459279793), (5706, 0.7205766921228921), (5791,
0.4385290096535153), (5825, 0.8600261451922278), (6057, 0.537086155529574), (6207, 0.5860090386731196), (6233,
-0.11720180773462363), (6369, 0.657793514480273), (6630, 0.4385290096535145), (6743, 0.8594395636904102), (6976,
0.06296909605425952), (6978, 0.43852900965351443), (7083, 0.3440104580768906), (7443, 0.8320502943378437), (7539,
0.6147413901342469), (7545, 0.15293388567865973), (8032, 0.804843636585534), (8082, 0.560448538317805), (8207,
0.716114874039432), (8316, 0.8204126541423654), (8405, 0.6577935144802716), (8412, 0.8648817040445187), (8417,
0.657793514480273), (8558, 0.6982240822943707), (8623, 0.8204126541423674), (8805, -0.43852900965351443), (8944,
0.7520710469952329), (8990, 0.6020183016345586), (9145, 0.11720180773462363), (9175, 0.5661385170722978), (9305,
0.7844645405527362), (9410, 0.89514359254929), (9454, 0.35805743701971815), (9650, 0.7032108464077436), (9772,
0.8951435925492954), (9790, -0.3827795011754764), (9795, 0.5853694070049635), (9838, 0.41602514716892186), (10012,
0.2256213140985701), (10059, 0.775890900167808), (10178, -0.23440361546924784), (10321, 0.0), (10387, 0.9460287597114506),
(10389, 0.4385290096535153), (10443, -0.323875137815648), (10497, 0.17902871850985907), (10514, 0.727218092353424), (10560,
0.8713121398246098), (10621, 0.4512426281971402), (10670, 0.29838119751643016), (10821, 0.22562131409856986), (10822,
0.04523081376574623), (10989, 0.5860090386731196)])
```

```
def create_pearson_df(pearson_corr_dict):
    pearson_df = pd.DataFrame.from_dict(pearson_corr_dict, orient='index')
    pearson_df.columns = ['similarityIndex']
    pearson_df['userId'] = pearson_df.index
    pearson_df.index = range(len(pearson_df))
    return pearson_df


pearson_df = create_pearson_df(pearson_corr_dict)
pearson_df.sample(5)
```

	similarityIndex	userId
19	0.897610	2047
31	0.864882	3269
85	0.225621	10012
72	-0.438529	8805
93	0.727218	10514

The top x similar users to input user

```
# Now let's get the top 50 users that are most similar to the input.
top_users = pearson_df.sort_values(by='similarityIndex', ascending=False)[0:50]
top_users.head()
```

	similarityIndex	userId
89	0.946029	10387
19	0.897610	2047
81	0.895144	9772
23	0.895144	2515
10	0.895144	1715

Now, let's start `recommending movies` to the input user.

Rating of selected users to all movies

We're going to do this by taking the `weighted average` of the ratings of the movies using the `Pearson Correlation` as the weight. But to do this, we first need to get the movies watched by the users in our pearson_df from the ratings dataframe and then store their correlation in a new column called `similarityIndex`. This is achieved below by merging of these two tables.

```
top_users_rating = top_users.merge(ratings_df, left_on='userId', right_on='userId', how='inner')
top_users_rating.head()
```

	similarityIndex	userId	movieId	rating
0	0.946029	10387	1	4.0
1	0.946029	10387	2	3.5
2	0.946029	10387	10	3.0
3	0.946029	10387	11	3.0
4	0.946029	10387	17	3.0

Now all we need to do is simply multiply the movie rating by its weight (The similarity index), then sum up the new ratings and divide it by the sum of the weights.

We can easily do this by simply multiplying two columns, then grouping up the dataframe by movieId and then dividing two columns.

It shows the idea of all similar users to candidate movies for the input user

```
# Multiplies the similarity by the user's ratings
top_users_rating['weightedRating'] = top_users_rating['similarityIndex'] * top_users_rating['rating']
top_users_rating.head()
```

	similarityIndex	userId	movieId	rating	weightedRating
0	0.946029	10387	1	4.0	3.784115
1	0.946029	10387	2	3.5	3.311101
2	0.946029	10387	10	3.0	2.838086
3	0.946029	10387	11	3.0	2.838086
4	0.946029	10387	17	3.0	2.838086

```
# Applies a sum to the top_users after grouping it up by userId
temp_top_users_rating = top_users_rating.groupby('movieId').sum()[['similarityIndex', 'weightedRating']]
temp_top_users_rating.columns = ['sum_similarityIndex', 'sum_weightedRating']
temp_top_users_rating.head()
```

movieId	sum_similarityIndex	sum_weightedRating
1	38.821238	146.424613
2	38.821238	101.191887
3	13.674659	35.392039
4	3.586101	9.326486
5	9.194413	23.109653

```
# Creates an empty dataframe
recommendation_df = pd.DataFrame()

# Now we take the weighted average
recommendation_df['weighted average recommendation score'] = temp_top_users_rating['sum_weightedRating'] / temp_top_users_ratin
recommendation_df['movieId'] = temp_top_users_rating.index
recommendation_df.head()
```

movieId	weighted average recommendation score	movieId
1	3.771766	1
2	2.606612	2
3	2.588148	3
4	2.600732	4
5	2.513445	5

```
# Now let's sort it and see the top 20 movies that the algorithm recommended!
recommendation_df = recommendation_df.sort_values(by='weighted average recommendation score', ascending=False)
recommendation_df.head(10)

movies_df.loc[movies_df['movieId'].isin(recommendation_df.head(10)['movieId'].tolist())]
```

	movieId	title	genres	year
1829	1913	Picnic at Hanging Rock	[Drama, Mystery]	1975